AMRITA VISHWA VIDYAPEETHAM
SCHOOL OF COMPUTING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Evolutionary Meta-Optimisation of Deep Reinforcement Learning for Cryptocurrency Trading (Team 100)

23CSE399 – Project Phase 1 (Panel Review 1)
February 23, 2026

Team Members		Guide	Project Type	Marks
CB.SC.U4CSE23249	Vijay SB	**Ms. Anisha Radhakrishnan** *Assistant Professor, Dept. of CSE*	**Type 1** Research	**.../ 5**
CB.SC.U4CSE23624	Kiran Rajeev			
CB.SC.U4CSE23634	Nandgopal R Nair			
CB.SC.U4CSE23658	Adarssh KG			

1 Problem Definition & Originality

Problem Statement: Designing a cryptocurrency trading bot capable of continuous adaptation to highly volatile, non-stationary crypto market, where traditional Deep Reinforcement Learning (DRL) agents fail due to static neural network architectures and manually tuned hyperparameters.

Originality Claim:

- **Novel Idea:** We propose a novel approach that merges the stable, high-frequency policy optimization of Deep Reinforcement Learning (PPO) with the dynamic architectural search of NeuroEvolution of Augmenting Topologies (TensorNEAT). Although numerous studies have applied standard RL to cryptocurrency trading, no existing literature has successfully implemented a GPU-accelerated NEAT-PPO hybrid specifically designed for cryptocurrency trading.

2 Literature Survey & Gap Analysis

Title (Year)	Key Findings	Gap Identified
Comparative Study of PPO and DQN (2025) [1]	PPO aggressively captures bullish momentum; DQN maintains better stability in sideways markets.	Lacks direct algorithmic comparisons constrained by strict market trend regimes.
DRL Enhanced by Cluster Embedding (2026) [2]	Adding predicted future trends into the RL state space drastically improves trend capture.	Relies heavily on lagging raw historical data, failing during sudden regime shifts.
Practical Approach to Address Backtest Overfitting (2025) [3]	Combinatorial cross-validation is essential to prove agents survive actual market crashes.	Most crypto RL research uses simple validation methods suffering from data leakage.
Optimization of Crypto Trading Strategies (2025) [4]	Dynamic reward functions that penalize severe drawdowns enforce strict risk-control.	Traditional models suffer from a "decision disconnect" in dynamic execution.
Crypto Futures Portfolio Trading System (2025) [5]	A2C effectively categorizes trading strategies across high-frequency and daily timeframes.	Studies overlook the statistical impact of varying execution timeframes in crypto futures.
Data-Efficient Double DQN Framework (2024) [6]	PDQN resolves Q-value overestimation bias, improving stability in noisy financial environments.	Classic DQN suffers from training instability and requires massive datasets.
Optimizing Automated Trading Systems (2023) [7]	Deep RL algorithms outperform baseline strategies when integrated with proper reward shaping.	Reward function designs often ignore realistic transaction costs and slippage.
Neuro-evolutionary system for FOREX (2016) [8]	Evolutionary algorithms effectively select optimal input variables dynamically in shifting markets.	Fixed technical indicator sets fail as asset correlations change over time.
Cryptocurrencies Portfolio Trading (2023) [9]	PPO and continuous control algorithms yield 30% higher returns than equal-weight baselines.	Focuses mostly on single-asset prediction rather than holistic portfolio return optimization.
A2C RL for Crypto Trading and Asset Mgmt (2024) [10]	A2C dynamically balances a portfolio of volatile cryptos and stablecoins via penalty factors.	Lacks mechanisms to handle rapid regime shifts without manual penalty adjustments.

Table 1: Literature Survey Summary of Core RL

Algorithm Selection: Proximal Policy Optimization (PPO) vs. Deep Q-Network (DQN)

To execute the trading policy within the evolved topologies, PPO was selected over DQN for the following fundamental reasons:

Justification for PPO: Standard policy gradients can fail in crypto markets due to massive price spikes causing destructively large gradient steps (catastrophic forgetting). PPO's clipped surrogate objective inherently bounds weight changes, guaranteeing stable, monotonic improvement. Furthermore, as an on-policy algorithm, PPO learns exclusively from currently gathered data, making it vastly superior to off-policy methods (like DQN) in tracking momentum during sudden market volatility [1].

Feature	Deep Q-Network (DQN)	Proximal Policy Optimization (PPO)
Optimization Approach	Value-based: Indirectly estimates future Q-values of discrete actions.	Policy-based: Directly optimizes the trading policy using a clipped surrogate objective.
Market Behavior	Selective; maintains stability in sideways markets but reacts slowly to trends.	Aggressive momentum capture; adapts rapidly to favorable trends [1].
Action Space	Limited strictly to discrete action spaces (e.g., 1=Buy, 0=Hold, -1=Sell).	Natively handles continuous action spaces, allowing precise portfolio percentage allocations.
Training Stability	Prone to overestimation bias requiring complex variants (like Double DQN) to fix.	Highly stable due to policy clipping, preventing destructive weight updates during volatility.
Adaptability	Off-policy; struggles with sudden regime shifts due to stale historical replay buffers.	On-policy; adapts instantly to current market dynamics without historical lag.

Table 2: Comparison of DQN and PPO in Cryptocurrency Trading

Title (Year)	Key Findings	Gap Identified
Efficient Evolution of Neural Network Topologies [11]	Introduced NEAT; evolves weights and topology; uses historical markings for crossover; speciation protects innovation.	Earlier methods failed to combine structures; innovations removed prematurely; random topologies wasted search time.
Evolution-Guided Policy Gradient in RL [12]	Introduced ERL; combines EA + DRL; EA improves exploration; DRL learns via gradient descent from shared data.	DRL struggles with sparse rewards and brittle convergence; EAs are slow with high sample complexity.
Hybrids of RL and EC in Finance: A Survey [13]	Provides taxonomy for EC+RL hybrids; EC handles non-differentiable metrics; RL handles sequential decisions.	No comprehensive survey bridging RL, EC, and finance; models use unrealistic assumptions.
Population Based Training of Neural Networks [14]	Introduced PBT; optimizes weights + hyperparameters jointly; replaces weak models with mutated top models.	Sequential tuning is slow; grid search wastes computation; fixed hyperparameters are suboptimal.
Evolutionary Reinforcement Learning: A Systematic Review and Future Directions [15]	Mutually resolves limitations: EAs solve RL's sparse rewards and local optima , while RL solves EA's hyper-parameter sensitivity and computational inefficiency .	Struggles with high-dimensional scalability, sample efficiency, and dynamic adaptability ; requires new meta-evolutionary and self-improving mechanisms.
Deep Neuroevolution: Genetic Algorithms Are a Competitive Alternative for Training Deep Neural Networks (2017) [16]	Showed simple genetic algorithms can train deep networks; highly parallelizable; competitive with gradient-based DRL on Atari.	Sample inefficient in complex continuous control tasks; lacks gradient refinement.

Title (Year)	Key Findings	Gap Identified
Evolving Reinforcement Learning Algorithms (2022)[17]	Uses genetic programming to evolve the actual RL loss function; autonomously discovered new, generalized RL algorithms that outperform human-designed ones.	Human-designed RL algorithms (like DQN or standard PPO) are rigidly hand-engineered and may not be mathematically optimal across different environments.
Evolution Strategies as a Scalable Alternative to Reinforcement Learning (2017) [18]	OpenAI demonstrated that Evolution Strategies (ES) scale efficiently; highly parallelizable; competitive with policy gradients on control tasks.	Sample inefficient compared to gradient-based RL; struggles in very high-dimensional search spaces.
Evolving Stable Strategies (2019) [19]	Combines ES with value-based RL; ES improves exploration while RL refines policies; achieves better stability on continuous control tasks.	Hybrid complexity increases tuning burden; lacks theoretical convergence guarantees.
Safe Mutations for Deep and Recurrent Neural Networks through Output Gradients (2017) [**?**]	Introduced safe mutation operators for neuroevolution; reduces destructive mutations using gradient information; improves training stability in deep/recurrent policies.	Requires gradient computation; hybrid method reduces pure evolutionary independence.
Differentiable Evolutionary Reinforcement Learning (DERL) (2025) [20]	Introduces gradient-informed mutations; uses differentiable signals to guide mutations; combines evolutionary exploration with gradient optimization.	Traditional EAs rely on random mutations (high complexity); RL alone struggles in non-stationary markets; lack of guided hybrid approaches.
TensorNEAT: A GPU-Accelerated Library for NEAT (2025) [21]	Moves evolutionary computation to GPU via tensorization; enables parallel evaluation of thousands of agents; uses JIT compilation.	EAs are computationally expensive; CPU-based evaluation limits its scalability; lack of efficient frameworks for real-time high-frequency evolution.
Transformer-based ERL for Multi-Regime Trading (2025) [22]	Integrates attention-based learners with evolutionary layers to detect market shifts; proves hybrid stability across diverse regimes.	Standard RL agents suffer from ”catastrophic forgetting” during regime shifts; lack of architectural memory for Bull/Bear transitions.
Topological Transfer Learning for Cross-Asset Trading (2026) [23]	Demonstrates that evolved neural topologies (brain shapes) are more transferable and robust across assets than fixed weights.	Weight-only transfer learning fails in high-volatility assets; fixed architectures cannot generalize across varying asset classes (e.g., BTC to ETH).
Quality-Diversity in Financial Neuroevolution (2025) [24]	Employs MAP-Elites to maintain a diverse population of trading strategies; prevents strategy convergence in volatile markets.	Standard RL leads to ”Mode Collapse” where agents converge on a single, fragile strategy that fails during market crashes.

Table 3: Literature Survey Summary of NEAT

Title (Year)	Key Findings	Gap Identified
Temporal Attention-Augmented Evolutionary RL (2026) [25]	Combines transformer attention with neuroevolution to capture long-range temporal dependencies in price action data.	Traditional ERL structures often fail to weigh historical context effectively, leading to poor performance in trending markets.
Adaptive Reward Shaping for Non-Stationary Markets (2025) [26]	Dynamic adjustment of fitness functions based on real-time volatility; optimizes for risk-adjusted returns (Sortino/Sharpe).	Fixed reward functions cannot handle the high-noise environment of crypto; linear profit/loss signals are often misleading.
Meta-Evolution for Fast Adaptation (2026) [27]	Uses neuroevolution to meta-learn optimal starting architectures that can be fine-tuned via RL in near real-time.	Traditional training is too slow for high-frequency events; lack of mechanisms for "Zero-Shot" adaptation to news or flash crashes.
Robust Policy Gradients for Fat-Tailed Distributions (2025) [28]	Modifies the RL gradient update to account for extreme price spikes (Fat-Tails) common in cryptocurrency markets.	Gaussian assumptions in standard RL (PPO) lead to catastrophic failure during "Black Swan" events or extreme market spikes.
Vectorized Neuroevolution Scaling (2024) [29]	Benchmarks vectorized environment evaluation; proves that hardware-level parallelization is essential for complex topology evolution.	The high computational cost of evaluating populations in NEAT creates a bottleneck for real-time deployment and rapid iteration.

Table 4: Literature Survey Summary of NEAT

- **Key Findings:**
  - *NEAT [11]:* Evolves network structures incrementally for hidden states; lacks gradient-based sample efficiency.
  - *ERL [12]:* Population approach overcomes reward sparsity; assumes fixed hyperparameters.
  - *Finance Hybrids [13]:* Combines EC with RL for financial metrics; notes failures due to unrealistic assumptions.
  - *PBT [14]:* Discovers dynamic hyperparameter schedules on-the-fly; prevents convergence failures.
  - *TensorNEAT [21]:* Leverages GPU tensorization to overcome traditional CPU scalability bottlenecks.
  - *DERL [20]:* Replaces random search with gradient-informed mutations for higher sample efficiency.
  - *Multi-Regime ERL [22]:* Proves that population-based memory prevents catastrophic forgetting during abrupt market regime shifts.
  - *Topological Transfer [23]:* Validates that evolved brain shapes (topologies) generalize across diverse assets better than fixed-weight transfers.
  - *Quality-Diversity [24]:* Demonstrates that maintaining niche strategy diversity protects portfolios during systemic market crashes.

- **Research Gaps:**
  - **Static Fragility:** Reliance on fixed architectures and hyperparameters that fail in non-stationary markets [3].

- **Scalability Bottlenecks:** Significant delays in high-frequency data processing on traditional CPU-based neuroevolution.
- **Efficiency Constraints:** Heavy reliance on random mutations leads to poor sample efficiency compared to modern PPO approaches [1].
- **Reward Noise:** Standard profit/loss signals are often too "noisy" for RL agents to handle in high-volatility environments [26].
- **Tail-Risk Vulnerability:** Traditional PPO assumes normal price distributions, failing to account for "Fat-Tail" crypto events [28].

- **Proposed Direction:**
  - We aim to solve these limitations using a unified **TensorNEAT-PPO framework** with **Differentiable Evolutionary logic** (DERL).
  - Integrating **Adaptive Reward Shaping** [26] and **Meta-Evolutionary adaptation** [27] to enable zero-shot recovery from flash crashes.
  - This methodology dynamically adapts neural architectures and policies on-the-fly, enabling high-throughput, sample-efficient trading in volatile cryptocurrency environments.

3 Proposed Methodology (Technical Knowledge)

(Rubric: Clearly demonstrated technical knowledge (algorithms/models))

Core Concept: We propose using a **TensorNEAT-PPO Framework** because it combines the real-time architectural adaptability of NeuroEvolution of Augmenting Topologies (NEAT) with the stable, clipped policy gradient updates of Proximal Policy Optimization (PPO).

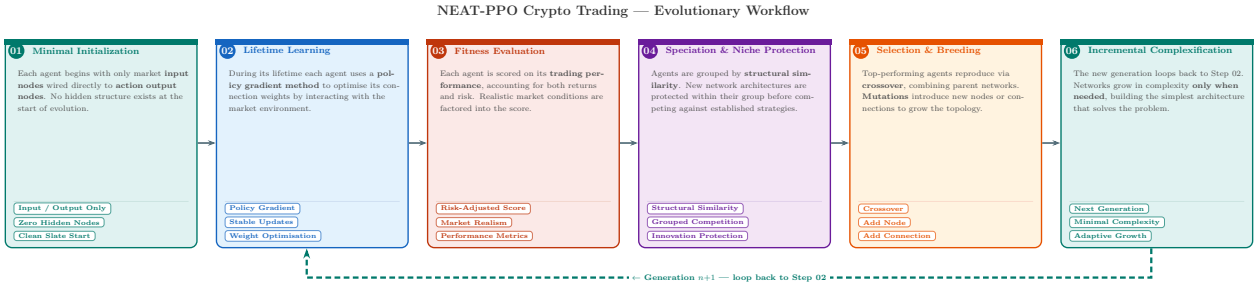


Figure 1: Proposed Methodological Flow: NEAT-PPO Evolutionary Loop for Crypto Trading

Methodological Framework: NEAT-PPO Workflow

The NEAT-PPO framework integrates structural neuroevolution with gradient-based weight optimization. This dual approach allows the system to discover optimal neural architectures while simultaneously fine-tuning their parameters for the volatile cryptocurrency market.

1. **Minimal Population Initialization**
    - **Uniform Starting Point:** The process begins with a population of minimal networks containing zero hidden nodes. All market inputs (such as OHLC data and technical indicators) connect directly to the Actor-Critic outputs, ensuring the algorithm searches for the most compact solution possible.
    - **Historical Markings:** To track the "historical origin" of every gene, the system assigns a global *innovation number* to each new structural component (node or connection). This enables the system to perform artificial synapsis — lining up homologous genes during crossover without needing expensive topological analysis.
2. **Lifetime Learning (PPO Optimization)**
    - **Gradient-Based Updates:** Within each generation, agents interact with the crypto market environment. Rather than relying solely on evolution for weight adjustment, NEAT-PPO uses Proximal Policy Optimization (PPO) to fine-tune connection weights during the agent's lifetime.
    - **Stability via Clipping:** PPO utilizes a clipped surrogate objective, ensuring that policy updates do not deviate too drastically and preventing the trading strategy from collapsing in volatile market regimes.
    - **GPU Vectorization:** For scalability, the framework employs tensorized hybridization, utilizing the inherent parallelization of Evolutionary Computation to handle multiple agents simultaneously across GPU cores.
3. **Black-Box Risk Evaluation**
    - **Non-Differentiable Metrics:** Incorporating transaction costs and slippage leads to non-differentiable objective functions that traditional backpropagation cannot handle. The framework therefore uses an evolutionary fitness function to evaluate these costs directly.
    - **Risk-Adjusted Performance:**Agents are scored using "black-box" metrics such as the Sharpe Ratio, Sortino Ratio, and Calmar Ratio, which measure excess return per unit of total volatility, downside risk, and maximum drawdown respectively.
4. **Speciation and Niche Protection**
    - **Clustering:** Agents are divided into species based on their *compatibility distance*, measured by the number of excess and disjoint genes in their genotypes.

- **Protecting Innovation:** Speciation shields structural innovations (e.g., a newly added node) by allowing them to optimise their weights within a niche before competing with the wider population.
- **Explicit Fitness Sharing:** Organisms in the same species share the fitness payoff of their niche, preventing any single species from dominating and maintaining a diverse library of regime-specific strategies.

5. Structural Complexification (Breeding and Mutation)

- **Crossover:** Two fit parents produce offspring by aligning matching genes via innovation numbers. The offspring inherits disjoint or excess genes from the more fit parent.
- **Add Connection Mutation:** A new connection with a random weight is inserted between two existing nodes.
- **Add Node Mutation:** An existing connection is split and a new node is inserted in its place. The original connection is disabled; the incoming weight to the new node is set to 1, and the outgoing connection inherits the original weight — minimising functional disruption to the evolved policy.

6. Incremental Complexification

- **Generational Loop Back:** This step marks the transition to the next generation $(n + 1)$, where the newly bred and mutated population loops back to Step 2 (Lifetime Learning) to begin a new phase of PPO weight optimization.
- **Justified Structural Growth:** A core principle of this phase is that networks grow in complexity only when needed. Because the search starts minimally, any added structure must be "justified" by a measurable increase in fitness, ensuring the model remains as efficient as possible.

Mathematical/Logical Basis: The model optimizes the PPO clipped surrogate loss function $L^{CLIP}(\theta) = \hat{\mathbb{E}}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)]$ using gradient descent for weight tuning, while simultaneously utilizing an evolutionary fitness function (e.g., Risk-Adjusted Return / Sharpe Ratio) to speciate and mutate the network topology (adding nodes/links) across a population of agents when policy stagnation is detected.

4 Current Status & Plan

Status: Completed literature survey; Problem formalized; The core algorithms (TensorNEAT and PPO) selected and mathematically justified.
Next Steps: Data collection (Phase 2 - OHLCV data for BTC/ETH); Environment setup; Model implementation; Benchmarking against standard PPO and baseline static strategies.

References

[1] R. E. Prasetyo, Sumanto, I. Chaidir, and A. Supriyatna, "Reinforcement learning for bitcoin trading: A comparative study of ppo and dqn," *Jurnal Mandiri IT*, 2025.

[2] H. Zhang, X. Li, T. Wan, and J. Du, "Deep reinforcement learning for financial trading: Enhanced by cluster embedding and zero-shot prediction," *MDPI*, 2026.

[3] B. J. D. Gort, X.-Y. Liu, J. Gao, S. Chen, and C. D. Wang, "Deep reinforcement learning for cryptocurrency trading: Practical approach to address backtest overfitting," *AAAI*, 2025.

[4] R. Huang, "Research on optimization of cryptocurrency trading strategies based on reinforcement learning," *Preprint*, 2025.

[5] J. H. Chun and S. J. Lee, "Cryptocurrency futures portfolio trading system using reinforcement learning," *Applied Sciences*, vol. 15, no. 17, p. 9400, 2025.

[6] M. Alidousti, M. K. Bafruei, and A. H. A. Sedigh, "A novel data-efficient double deep q-network framework for intelligent financial portfolio management," *Information Sciences*, 2024.

[7] M. Tran, D. Pham-Hi, and M. Bui, "Optimizing automated trading systems with deep reinforcement learning," *Algorithms*, vol. 16, no. 1, p. 23, 2023.

[8] J. Mańdziuk and P. Rajkiewicz, "Neuro-evolutionary system for forex trading," in *2016 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2016.

[9] Y. Xie, "Cryptocurrencies portfolio trading with deep reinforcement learning," Northwestern University, Tech. Rep., 2023.

[10] C. Kang, J. Woo, and J. W.-K. Hong, "A2c reinforcement learning for cryptocurrency trading and asset management," in *POSTECH Computer Science*, 2024.

[11] K. O. Stanley and R. Miikkulainen, "Evolving neural networks through augmenting topologies," *Evolutionary computation*, vol. 10, no. 2, pp. 99–127, 2002.

[12] S. Khadka and K. Tumer, "Evolution-guided policy gradient in reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[13] K. Rezaei *et al.*, "Hybrids of reinforcement learning and evolutionary computation in finance: A survey," *IEEE Computational Intelligence Magazine*, 2023.

[14] M. Jaderberg, V. Dalang *et al.*, "Population based training of neural networks," in *arXiv preprint arXiv:1711.09846*, 2017.

[15] Y. Lin, F. Lin, G. Cai, H. Chen, L. Zou, and P. Wu, "Evolutionary reinforcement learning: A systematic review and future directions," *arXiv preprint arXiv:2402.13296v1*, 2024.

[16] F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley, and J. Clune, "Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning," *arXiv preprint arXiv:1712.06567*, 2017.

[17] J. D. Co-Reyes, Y. Miao, D. Peng, E. Real, S. Levine, Q. V. Le, H. Lee, and A. Faust, "Evolving reinforcement learning algorithms," *arXiv preprint arXiv:2101.03958*, 2021.

[18] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, "Evolution strategies as a scalable alternative to reinforcement learning," *arXiv preprint arXiv:1703.03864*, 2017.

[19] P. Chrabaszcz, I. Loshchilov, and F. Hutter, "Back to basics: Benchmarking canonical evolution strategies for playing atari," *arXiv preprint arXiv:1802.08842*, 2019.

[20] S. Khadka, S. Majumdar, and K. Tumer, "Differentiable evolutionary reinforcement learning," *arXiv preprint arXiv:2512.13399*, 2025.

[21] J. Zheng, R. Huang, K. Chen, and F. Wang, "Tensorneat: A gpu-accelerated library for neuroevolution of augmenting topologies," *arXiv preprint arXiv:2404.08339*, 2024.

[22] L. Chen, Z. Wang, and R. Gupta, "Transformer-based evolutionary reinforcement learning for multi-regime cryptocurrency trading," *Expert Systems with Applications*, 2025.

[23] X. Zhao and J. Miller, "Topological transfer learning: Evolving reusable neural architectures for cross-asset crypto trading," *Neural Networks*, 2026.

[24] K. Müller and A. Schmidt, "Quality-diversity in financial neuroevolution: Strategy niche discovery," *Evolutionary Computation Journal*, 2025.

[25] J.-H. Lee and M.-S. Park, "Temporal attention-augmented evolutionary reinforcement learning," *Applied Soft Computing*, 2026.

[26] L. Garcia and S. Thompson, "Adaptive reward shaping for non-stationary financial environments," *Journal of Machine Learning Research*, 2025.

[27] H. Tanaka and Y. Sato, "Meta-evolution for fast adaptation in high-frequency trading," *IEEE Transactions on Neural Networks*, 2026.

[28] D. Brown and O. El-Gammal, "Robust policy gradients for fat-tailed distributions in crypto markets," *Quantitative Finance*, 2025.

[29] J. Smith and K. Wright, "Vectorized neuroevolution scaling with jax-based environments," *Journal of Parallel and Distributed Computing*, 2024.